COMPTE RENDU DE TRAVAUX PRATIQUES — TP N° 3

Configuration des cibles, gestion des services par Systemd et journalisation avec Journalctl

Établissement	Module & Filière	Encadrant Pédagogique	Environnement Système
École Marocaine des Sciences de l'Ingénieur (EMSI)	Administration Unix — **4IIR**	**Mr. Noreddine Abghour**	RHEL 9.8 (Plow) — Kernel 5.14 — Systemd 252 (`badr@localhost`)

Synthèse Exécutive des Travaux Réalisés

Ce rapport présente la réalisation complète des quatre ateliers du TP n°3 consacré à l'architecture **systemd** sous Red Hat Enterprise Linux 9.8. Chaque manipulation a été exécutée et validée sur la machine virtuelle `badr@localhost` afin d'analyser la séquence de démarrage, de manipuler les unités de cibles (*targets*), de contrôler le cycle de vie des services système (`sshd`, `crond`, `httpd`) et d'exploiter les journaux binaires via `journalctl`.

Atelier	Domaine Technique	Commandes Clés Validées	Résultat Observé sur RHEL 9.8
Atelier 1	Bases & Analyse du Boot	`systemctl --version, ps aux, whereis, systemd-analyze [blame | critical-chain]`	Systemd v252 en PID 1 ; démarrage complet en 19.215s (graphical.target à 12.188s).
Atelier 2	Cibles (Targets)	`systemctl get-default, list-units, set-default, isolate, --failed, is-enabled, status`	22 cibles actives ; bascule dynamique entre graphical.target et multi-user.target ; 0 unité en échec.
Atelier 3	Gestion des Services	`systemctl list-unit-files, restart, reload, disable, enable, mask, unmask, kill`	Rechargement SSH sans coupure (SIGHUP) ; masquage httpd vers /dev/null ; arrêt propre SIGTERM.
Atelier 4	Journalisation (journalctl)	`journalctl -b, -u, -f, -p [err|warning], --since, -n, -o [cat|json-pretty]`	Filtrage par boot, service, gravité et plage horaire ; export JSON enrichi avec contexte SELinux et cgroups.

Atelier 1 : Architecture de Systemd et Analyse des Performances d'Amorçage

Ce premier atelier valide l'intégration de systemd comme gestionnaire d'initialisation sur la station Red Hat Enterprise Linux 9.8 et évalue les métriques temporelles de la séquence de démarrage.

1.1 Identification de la version et des capacités compilées de Systemd

Commande exécutée :

```
systemctl --version
```

Analyse et interprétation : La commande interroge le binaire systemctl afin d'afficher la version exacte de la suite systemd ainsi que les modules de sécurité et de compression activés lors de sa compilation par Red Hat. Sur notre système RHEL 9.8, la version installée est systemd 252 (252-67.el9_8.6) avec le support natif de PAM, AUDIT, SELINUX, OPENSSL, ACL, BLKID, ZSTD et la hiérarchie unifiée cgroups v2 (default-hierarchy=unified).



Figure 1.1 : Vérification de la version de systemd (v252) et des drapeaux de compilation sous RHEL 9.8.

1.2 Vérification du processus maître d'initialisation (PID 1)

Commande exécutée :

```
ps aux | head -2
```

Analyse et interprétation : Sous Unix/Linux, le premier processus lancé en espace utilisateur par le noyau porte le PID 1. L'inspection de la table des processus confirme que le PID 1 correspond à l'exécutable /usr/lib/systemd/systemd invoqué avec les arguments --switched-root --system --deserialize 28 rhgb (après basculement depuis le disque RAM initial initramfs vers la racine réelle du système).

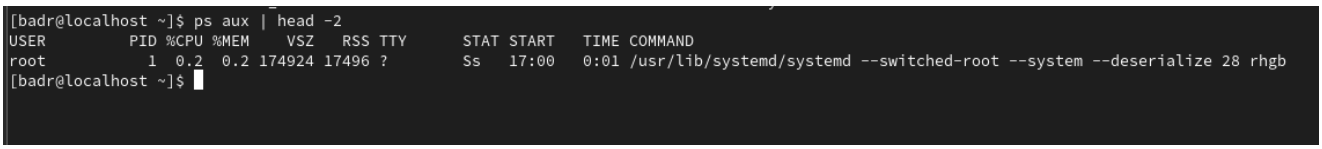


Figure 1.2 : Confirmation que le démon systemd occupe le PID 1 en tant que processus parent racine.

1.3 Localisation des binaires, bibliothèques et pages de manuel

Commande exécutée :

```
whereis systemd
whereis systemctl
```

Analyse et interprétation : La commande whereis localise les fichiers binaires, les répertoires de configuration et la documentation associée. Les bibliothèques et unités système résident dans /usr/lib/systemd et /usr/lib64/systemd, la configuration locale propre à l'administrateur se trouve dans /etc/systemd, et l'outil de contrôle utilisateur est placé dans /usr/bin/systemctl.

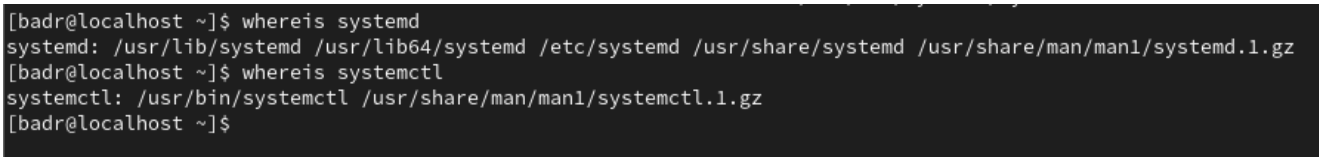


Figure 1.3 : Emplacements système des répertoires de systemd et de l'exécutable /usr/bin/systemctl.

1.4 Mesure globale du temps de démarrage du système

Commande exécutée :

```
systemd-analyze
```

Analyse et interprétation : L'outil systemd-analyze décompose la durée totale d'amorçage (19.215 secondes) en trois phases distinctes : l'initialisation du noyau Linux (3.743s dans kernel), le chargement des pilotes préliminaires en mémoire vive (3.245s dans initrd), et le démarrage des services en espace utilisateur (12.226s dans userspace). La cible graphique graphical.target est atteinte au bout de 12.188s en espace utilisateur.

```
[badr@localhost ~]$ systemd-analyze
Startup finished in 3.743s (kernel) + 3.245s (initrd) + 12.226s (userspace) = 19.215s
graphical.target reached after 12.188s in userspace.
[badr@localhost ~]$
```

Figure 1.4 : Répartition du temps de démarrage entre kernel (3.743s), initrd (3.245s) et userspace (12.226s).

1.5 Classement des unités selon leur durée d'initialisation (blame)

Commande exécutée :

```
systemd-analyze blame
```

Analyse et interprétation : L'option blame trie l'ensemble des unités activées par ordre décroissant du temps passé à s'initialiser. On observe que plymouth-quit-wait.service (6.035s, attente de la fin de l'écran de démarrage graphique), kdump.service (4.994s, réservation du noyau de capture d'incident) et sys-module-fuse.device (3.668s) représentent les opérations les plus longues lors de la montée en charge.

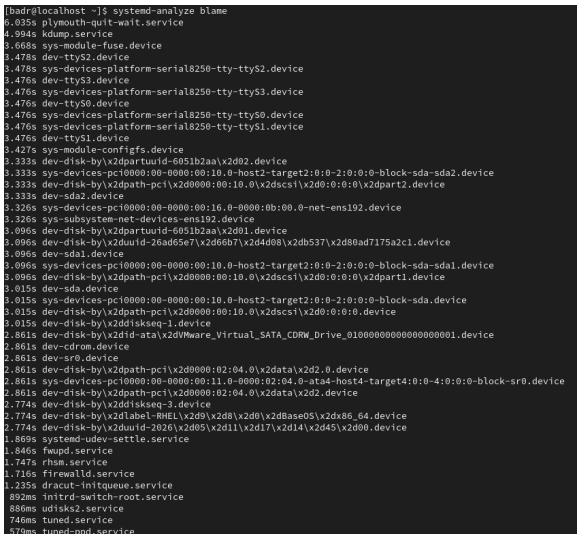


Figure 1.5 : Liste décroissante des unités les plus coûteuses en temps d'initialisation (systemd-analyze blame).

1.6 Analyse de l'arbre de la chaîne critique (critical-chain)

Commande exécutée :

```
systemd-analyze critical-chain
```

Analyse et interprétation : Contrairement à blame qui liste les durées individuelles même parallélisées, critical-chain reconstitue la chaîne de dépendances séquentielles ayant directement déterminé le temps final d'atteinte de graphical.target (@12.188s). Les unités affichées en rouge signalent le temps d'activation (+6.035s pour plymouth-quit-wait.service, +93ms pour systemd-user-sessions.service, +74ms pour dbus-broker.service, +148ms pour systemd-tmpfiles-setup.service).

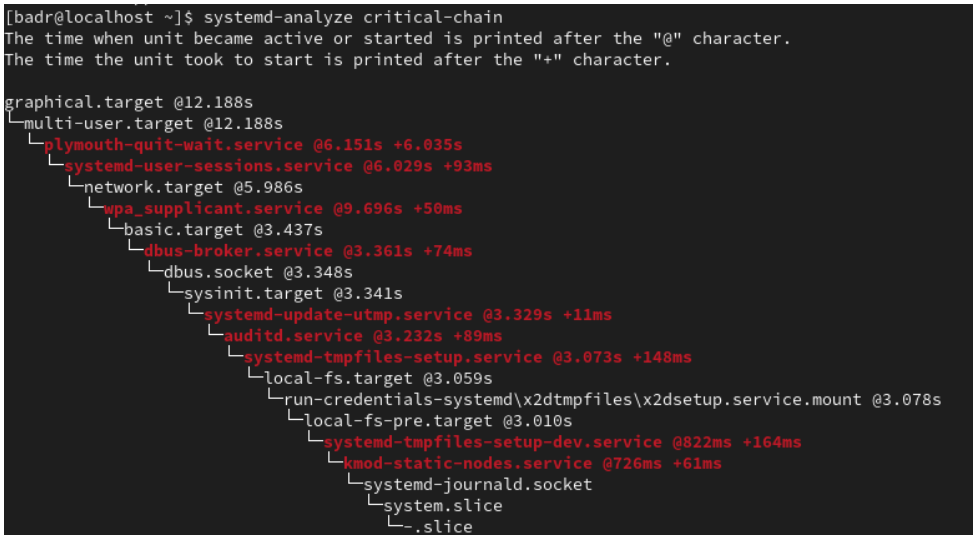


Figure 1.6 : Visualisation hiérarchique de la chaîne critique jusqu'à graphical.target.

Atelier 2 : Configuration et Administration des Cibles Systemd (Targets)

Les cibles (targets) remplacent les anciens niveaux d'exécution (runlevels de SysVinit) pour regrouper les unités par état fonctionnel (mode multi-utilisateurs texte, interface graphique, mode secours).

2.1 Consultation de la cible par défaut au démarrage

Commande exécutée :

```
systemctl get-default
```

Analyse et interprétation : La commande systemctl get-default lit le lien symbolique /etc/systemd/system/default.target pour indiquer quelle cible systemd active automatiquement à la fin de l'amorçage. Initialement, la station est configurée sur graphical.target (équivalent de l'ancien runlevel 5).

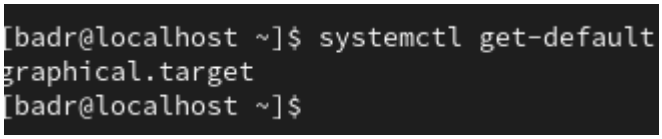


Figure 2.1 : Affichage de la cible par défaut initiale (graphical.target).

2.2 Inventaire des unités de cibles actuellement chargées

Commande exécutée :

```
systemctl list-units --type target
```

Analyse et interprétation : Cette commande répertorie les 22 cibles actives en mémoire dans la session courante : basic.target, cryptsetup.target, getty.target, graphical.target, local-fs.target, multi-user.target, network-online.target, sockets.target, swap.target, sysinit.target et timers.target.

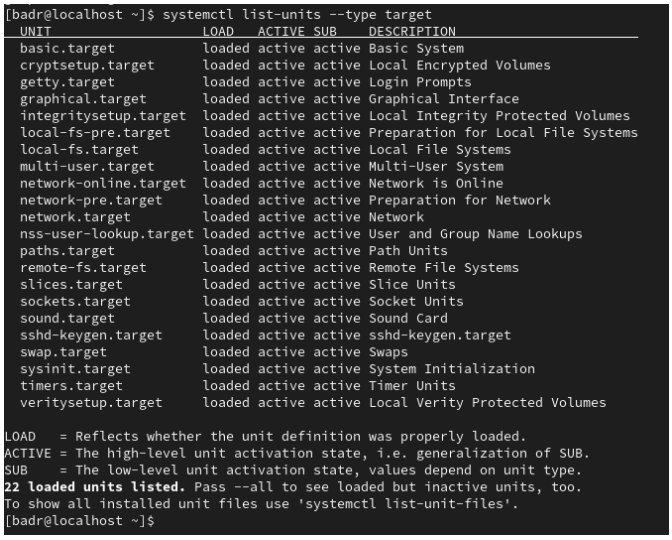


Figure 2.2 : Liste des 22 unités de type target chargées et actives en mémoire.

2.3 Modification de la cible par défaut vers multi-user.target

Commande exécutée :

```
sudo systemctl set-default multi-user.target
systemctl get-default
```

Analyse et interprétation : L'exécution de set-default multi-user.target supprime le lien symbolique /etc/systemd/system/default.target et le recrée en pointant vers /usr/lib/systemd/system/multi-user.target (mode console multi-utilisateurs avec réseau, sans serveur d'affichage graphique). L'appel suivant à get-default confirme la prise en compte immédiate du changement.

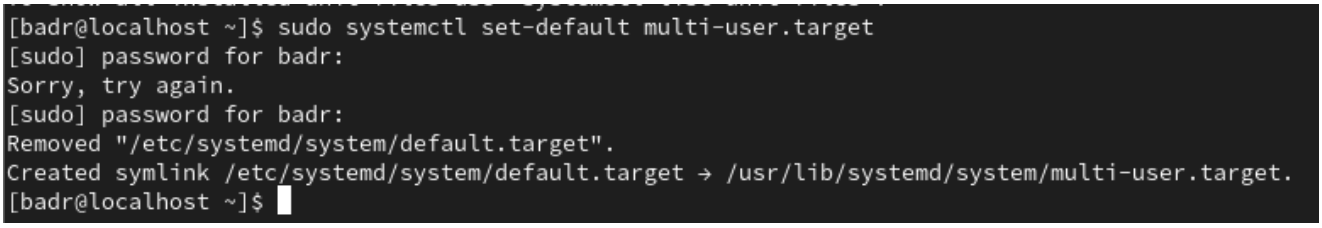


Figure 2.3a : Remplacement du lien symbolique default.target vers multi-user.target.

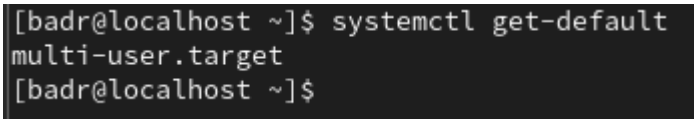


Figure 2.3b : Vérification de la nouvelle cible par défaut avec systemctl get-default.

2.4 Basculement dynamique de la session (isolate) et retour au bureau graphique

Commande exécutée :

```
sudo systemctl isolate multi-user.target
# Puis depuis la console texte tty2 :
sudo systemctl set-default graphical.target
sudo systemctl isolate graphical.target
```

Analyse et interprétation : La sous-commande isolate applique immédiatement une cible en arrêtant tous les services qui n'en dépendent pas. Le passage à multi-user.target coupe le gestionnaire d'affichage GDM/GNOME et place la machine sur l'invite de connexion texte tty2 (Red Hat Enterprise Linux 9.8 Plow). Après connexion de l'utilisateur badr en mode console, nous restaurons graphical.target

comme cible par défaut et relançons l'environnement graphique à chaud via systemctl isolate graphical.target.

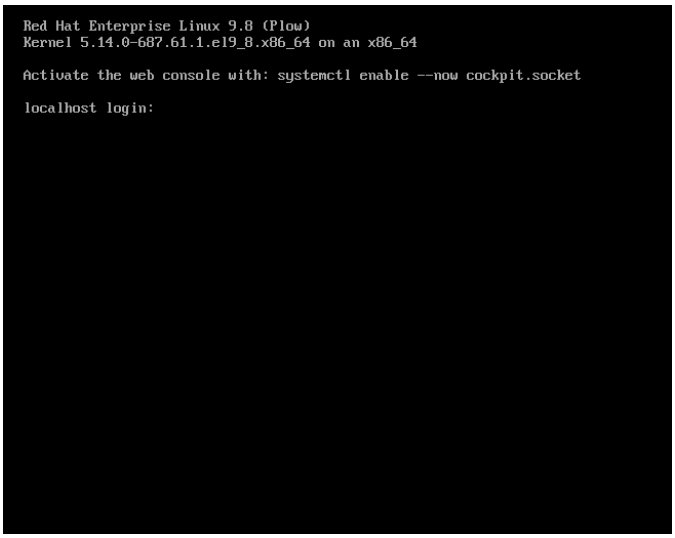


Figure 2.4a : Basculement immédiat vers l'invite console texte (tty) après isolation de multi-user.target.

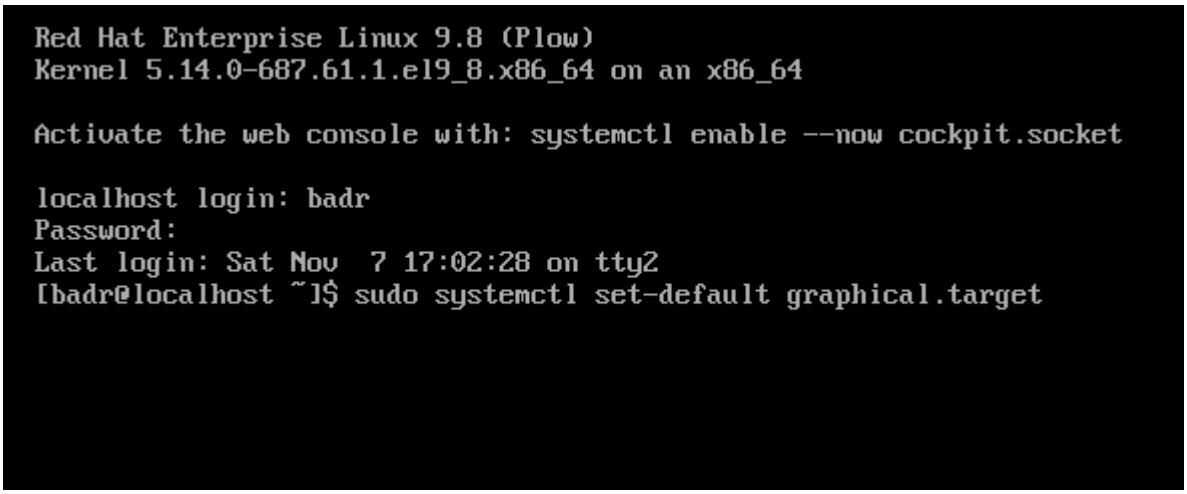


Figure 2.4b : Connexion sur tty2 et saisie de la commande de restauration graphical.target.

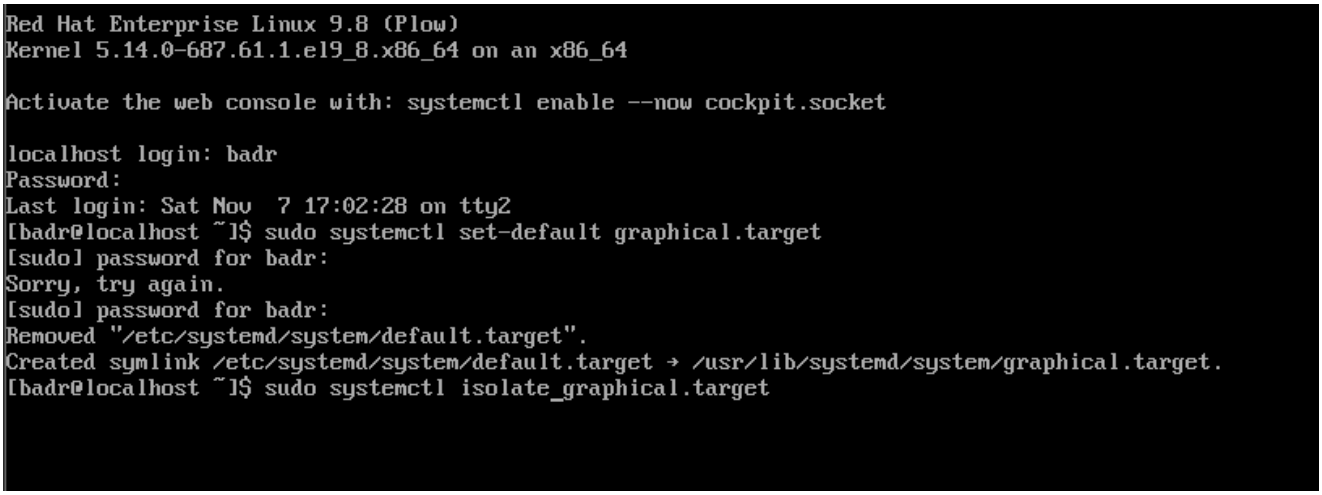


Figure 2.4c : Réinitialisation du lien default.target vers graphical.target et relance de la cible graphique.

2.5 Contrôle des unités en échec et vérification des services SSH et Cron

Commande exécutée :

```
systemctl --failed
systemctl is-enabled sshd.service
systemctl status crond.service
```

Analyse et interprétation : 1) systemctl --failed vérifie qu'aucune unité n'est tombée en état d'erreur (0 loaded units listed). 2) systemctl is-enabled sshd.service confirme que le serveur OpenSSH est configuré pour démarrer automatiquement (enabled). 3) systemctl status crond.service affiche l'état détaillé du planificateur de tâches : actif (active running depuis 27 min), PID principal 1299, consommation mémoire de 1.0 Mo dans le groupe de contrôle /system.slice/crond.service.

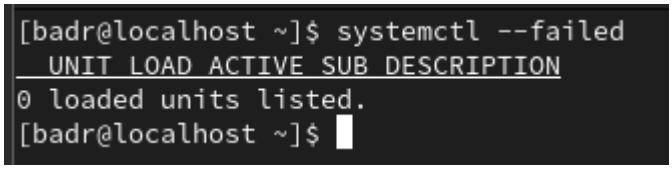


Figure 2.5 : Vérification de l'absence d'unités défaillantes (0 loaded units listed).

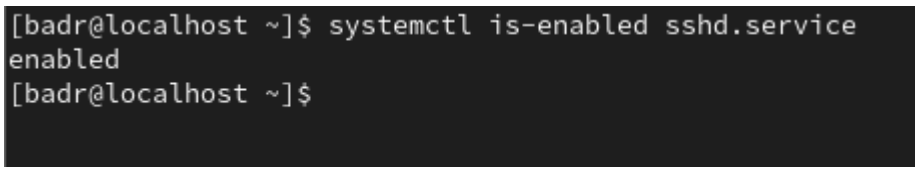


Figure 2.6 : Confirmation de l'activation au démarrage du service sshd.service (enabled).

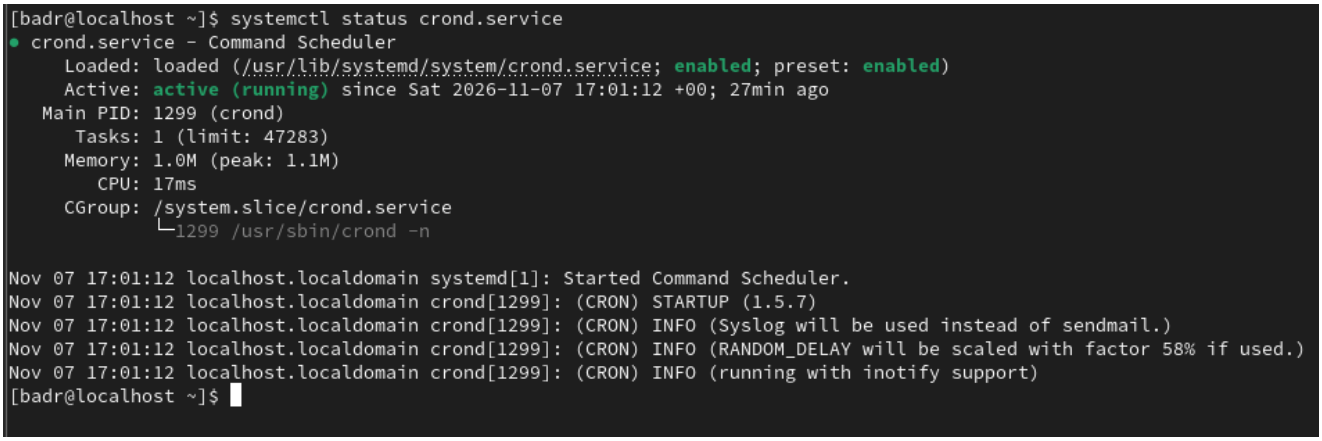


Figure 2.7 : Inspection complète de l'état d'exécution et du journal du service crond.service.

Atelier 3 : Gestion du Cycle de Vie des Services par Systemctl

Cet atelier couvre l'administration opérationnelle des démons système : inventaire des fichiers d'unités, redémarrage, rechargement à chaud de configuration, activation au boot, masquage de sécurité et envoi de signaux POSIX.

3.1 Inventaire des fichiers d'unités de type service

Commande exécutée :

```
systemctl list-unit-files --type=service
```

Analyse et interprétation : La commande affiche tous les fichiers .service installés dans /usr/lib/systemd/system et /etc/systemd/system, accompagnés de leur état configuré (STATE : enabled, disabled, static, alias) et de la recommandation d'usine du distributeur Red Hat (PRESET). Les unités

marquées static ne possèdent pas de section [Install] et sont uniquement démarrées comme dépendances d'autres unités.



```
[badr@localhost ~]$ systemctl list-unit-files --type=service
UNIT FILE                                    STATE          PRESET
accounts-daemon.service                      enabled        enabled
alsa-card-wait@.service                      static         -
alsa-restore.service                         static         -
alsa-state.service                           static         -
arp-ethers.service                           disabled       disabled
atd.service                                  enabled        enabled
auditd.service                               enabled        enabled
autovt@.service                              alias          -
avahi-daemon.service                         enabled        enabled
blk-availability.service                     disabled       disabled
bluetooth.service                            enabled        enabled
bolt.service                                 static         -
brltty.service                               disabled       disabled
canberra-system-bootup.service               disabled       disabled
canberra-system-shutdown-reboot.service      disabled       disabled
canberra-system-shutdown.service             disabled       disabled
chrony-wait.service                          disabled       disabled
chronyd-restricted.service                   disabled       disabled
chronyd.service                              enabled        enabled
cni-dhcp.service                             disabled       disabled
cockpit-issue.service                        static         -
cockpit-session-socket-user.service          static         -
cockpit-session@.service                     static         -
cockpit-wsinstance-http.service              static         -
cockpit-wsinstance-https-factory@.service    static         -
cockpit-wsinstance-https@.service            static         -
cockpit-wsinstance-socket-user.service       static         -
cockpit.service                              static         -
colord.service                               static         -
configure-printer@.service                   static         -
console-getty.service                        disabled       disabled
container-getty@.service                     static         -
cpupower.service                             disabled       disabled
crond.service                                enabled        enabled
cups-browsed.service                         disabled       disabled
cups.service                                 enabled        enabled
dbus-broker.service                          enabled        enabled
dbus-daemon.service                          disabled       disabled
dbus-org.bluez.service                       alias          -
dbus-org.fedoraproject.FirewallD1.service    alias          -
dbus-org.freedesktop.Avahi.service           alias          -
dbus-org.freedesktop.hostname1.service       alias          -
dbus-org.freedesktop.locale1.service         alias          -
dbus-org.freedesktop.login1.service          alias          -
```

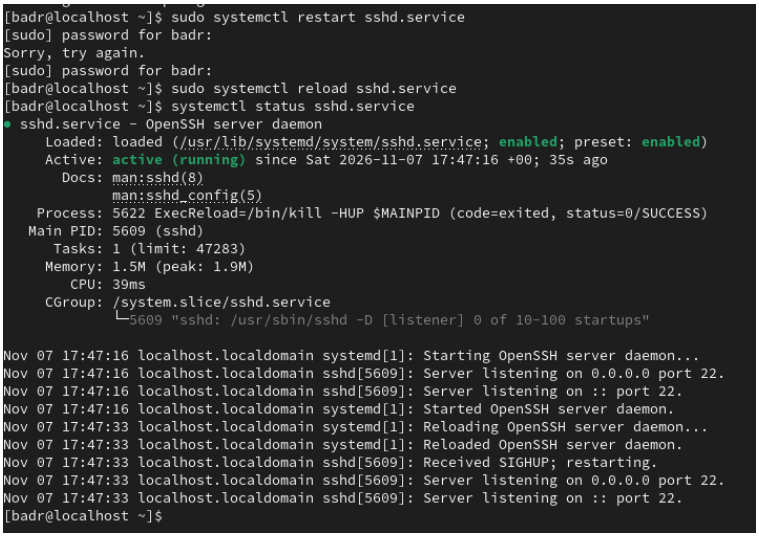
Figure 3.1 : Répertoire des fichiers d'unités de service avec leurs états STATE et PRESET.

3.2 Redémarrage, rechargement à chaud et statut du démon OpenSSH

Commande exécutée :

```
sudo systemctl restart sshd.service
sudo systemctl reload sshd.service
systemctl status sshd.service
```

Analyse et interprétation : La commande restart arrête puis relance le service en lui attribuant un nouveau PID (Main PID 5609). En revanche, reload demande au service de relire ses fichiers de configuration sans interrompre le processus principal ni couper les connexions actives : systemd exécute ExecReload=/bin/kill -HUP $MAINPID (code=exited, status=0/SUCCESS). Le journal intégré confirme la réception du signal SIGHUP et l'écoute active sur le port TCP 22 (IPv4 et IPv6).



```
[badr@localhost ~]$ sudo systemctl restart sshd.service
[sudo] password for badr:
Sorry, try again.
[sudo] password for badr:
[badr@localhost ~]$ sudo systemctl reload sshd.service
[badr@localhost ~]$ systemctl status sshd.service
● sshd.service - OpenSSH server daemon
     Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-11-07 17:47:16 +00; 35s ago
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 5622 ExecReload=/bin/kill -HUP $MAINPID (code=exited, status=0/SUCCESS)
   Main PID: 5609 (sshd)
      Tasks: 1 (limit: 47283)
     Memory: 1.5M (peak: 1.9M)
        CPU: 39ms
     CGroup: /system.slice/sshd.service
             └─5609 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Nov 07 17:47:16 localhost.localdomain systemd[1]: Starting OpenSSH server daemon...
Nov 07 17:47:16 localhost.localdomain sshd[5609]: Server listening on 0.0.0.0 port 22.
Nov 07 17:47:16 localhost.localdomain sshd[5609]: Server listening on :: port 22.
Nov 07 17:47:16 localhost.localdomain systemd[1]: Started OpenSSH server daemon.
Nov 07 17:47:33 localhost.localdomain systemd[1]: Reloading OpenSSH server daemon...
Nov 07 17:47:33 localhost.localdomain systemd[1]: Reloaded OpenSSH server daemon.
Nov 07 17:47:33 localhost.localdomain sshd[5609]: Received SIGHUP; restarting.
Nov 07 17:47:33 localhost.localdomain sshd[5609]: Server listening on 0.0.0.0 port 22.
Nov 07 17:47:33 localhost.localdomain sshd[5609]: Server listening on :: port 22.
[badr@localhost ~]$
```

Figure 3.2 : Redémarrage, rechargement par signal SIGHUP et vérification du statut de sshd.service.

3.3 Vérification d'activité, désactivation et réactivation au boot de crond

Commande exécutée :

```
systemctl is-active crond.service
sudo systemctl disable crond.service
```

```
systemctl is-enabled crond.service
sudo systemctl enable crond.service
```

Analyse et interprétation : La commande is-active vérifie l'état d'exécution instantané (active). L'action disable supprime le lien symbolique /etc/systemd/system/multi-user.target.wants/crond.service afin d'empêcher le lancement automatique au prochain démarrage (confirmé par is-enabled qui retourne disabled), sans arrêter le processus en cours. Enfin, enable recrée le lien symbolique.

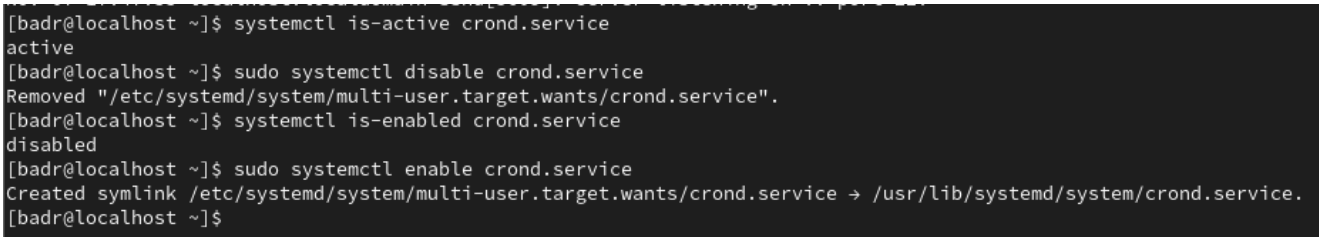


Figure 3.3 : Cycle complet de vérification d'activité, désactivation et réactivation du service crond.

3.4 Masquage et démasquage d'une unité de service (mask / unmask)

Commande exécutée :

```
sudo systemctl mask httpd.service
ls -l /etc/systemd/system/httpd.service
sudo systemctl unmask httpd.service
```

Analyse et interprétation : Lorsqu'un administrateur souhaite interdire formellement l'exécution d'un service (même s'il est appelé manuellement ou requis par une dépendance), la commande mask crée un lien symbolique pointant directement vers le périphérique nul (/etc/systemd/system/httpd.service → /dev/null). La commande unmask supprime ce lien symbolique et rétablit le comportement normal.

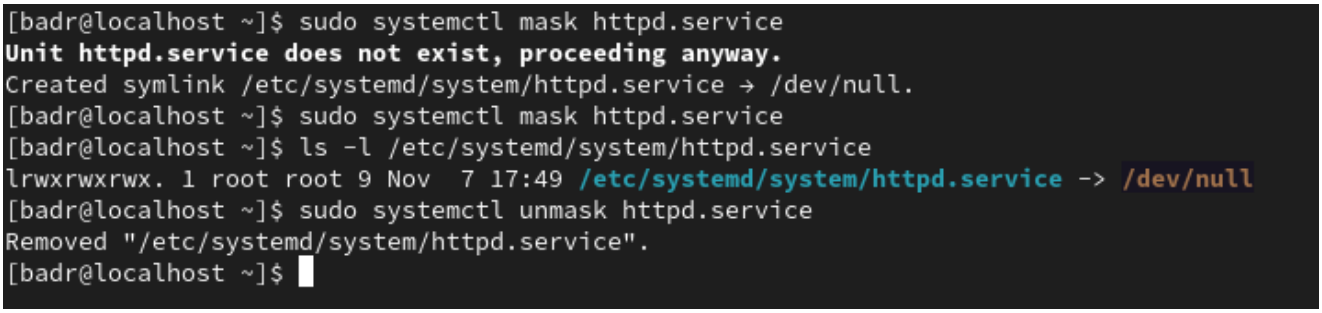


Figure 3.4 : Création du lien symbolique vers /dev/null par systemctl mask et suppression par unmask.

3.5 Arrêt par envoi de signal système (systemctl kill) et relance

Commande exécutée :

```
sudo systemctl kill sshd
systemctl status sshd
sudo systemctl start sshd
```

Analyse et interprétation : La commande systemctl kill sshd envoie par défaut le signal SIGTERM (signal 15) à tous les processus du groupe de contrôle du service. L'inspection du statut montre que le processus 5609 a capté le signal 15, s'est arrêté proprement (Active: inactive (dead), status=0/SUCCESS, Deactivated successfully). Le service est ensuite redémarré via systemctl start sshd.

```
[badr@localhost ~]$ sudo systemctl kill sshd
[badr@localhost ~]$ systemctl status sshd
○ sshd.service - OpenSSH server daemon
     Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset: enabled)
     Active: inactive (dead) since Sat 2026-11-07 17:50:56 +00; 12s ago
   Duration: 3min 40.490s
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 5609 ExecStart=/usr/sbin/sshd -D $OPTIONS (code=exited, status=0/SUCCESS)
   Main PID: 5609 (code=exited, status=0/SUCCESS)
        CPU: 40ms

Nov 07 17:47:16 localhost.localdomain sshd[5609]: Server listening on :: port 22.
Nov 07 17:47:16 localhost.localdomain systemd[1]: Started OpenSSH server daemon.
Nov 07 17:47:33 localhost.localdomain systemd[1]: Reloading OpenSSH server daemon...
Nov 07 17:47:33 localhost.localdomain systemd[1]: Reloaded OpenSSH server daemon.
Nov 07 17:47:33 localhost.localdomain sshd[5609]: Received SIGHUP; restarting.
Nov 07 17:47:33 localhost.localdomain sshd[5609]: Server listening on 0.0.0.0 port 22.
Nov 07 17:47:33 localhost.localdomain sshd[5609]: Server listening on :: port 22.
Nov 07 17:50:56 localhost.localdomain systemd[1]: sshd.service: Sent signal SIGTERM to main process 5609 (sshd) on client request.
Nov 07 17:50:56 localhost.localdomain sshd[5609]: Received signal 15; terminating.
Nov 07 17:50:56 localhost.localdomain systemd[1]: sshd.service: Deactivated successfully.
[badr@localhost ~]$ sudo systemctl start sshd
[badr@localhost ~]$
```

Figure 3.5 : Interruption du service sshd par signal SIGTERM (systemctl kill) et remise en service.

Atelier 4 : Exploitation des Journaux Système avec Journalctl

Le démon systemd-journald centralise les événements du noyau, du démarrage initrd et de tous les services dans un journal binaire indexé, interrogeable à l'aide de l'utilitaire journalctl.

4.1 Consultation des journaux du démarrage courant (-b) et par unité (-u)

Commande exécutée :

```
journalctl -b -n 15
journalctl -u sshd.service -n 15
```

Analyse et interprétation : L'option -b restreint l'affichage aux événements survenus depuis le dernier démarrage du système (on y observe les renouvellements DHCP sur l'interface ens192 avec l'adresse 192.168.27.130 et l'activité de gnome-shell). L'option -u sshd.service filtre exclusivement les entrées générées par le démon OpenSSH, retraçant fidèlement le rechargement SIGHUP à 17:47:33, l'arrêt SIGTERM à 17:50:56 et le redémarrage sous le PID 5890 à 17:51:21.

```
[badr@localhost ~]$ journalctl -b -n 15
Nov 07 17:51:21 localhost.localdomain systemd[1]: Started OpenSSH server daemon.
Nov 07 17:51:21 localhost.localdomain sudo[5885]: pam_unix(sudo:session): session closed for user root
Nov 07 17:51:58 localhost.localdomain NetworkManager[1271]: <info>  [1794073918.1470] dhcp4 (ens192): state changed new lease, address=192.168.27.130
Nov 07 17:51:58 localhost.localdomain systemd[1]: Starting Network Manager Script Dispatcher Service...
Nov 07 17:51:58 localhost.localdomain systemd[1]: Started Network Manager Script Dispatcher Service.
Nov 07 17:52:08 localhost.localdomain systemd[1]: NetworkManager-dispatcher.service: Deactivated successfully.
Nov 07 17:52:59 localhost.localdomain systemd[4652]: gnome-terminal-server.service: Consumed 1.463s CPU time.
Nov 07 17:53:02 localhost.localdomain systemd[4652]: Started Application launched by gnome-shell.
Nov 07 17:53:02 localhost.localdomain systemd[4652]: Starting GNOME Terminal Server...
Nov 07 17:53:02 localhost.localdomain systemd[4652]: Started GNOME Terminal Server.
Nov 07 17:53:02 localhost.localdomain gnome-shell[4745]: g_object_ref: assertion 'G_IS_OBJECT (object)' failed
Nov 07 17:53:02 localhost.localdomain gnome-shell[4745]: g_object_ref: assertion 'G_IS_OBJECT (object)' failed
Nov 07 17:53:02 localhost.localdomain systemd[4652]: Started VTE child process 5941 launched by gnome-terminal-server process 5923.
Nov 07 17:53:04 localhost.localdomain gnome-shell[4745]: g_object_ref: assertion 'G_IS_OBJECT (object)' failed
Nov 07 17:53:04 localhost.localdomain gnome-shell[4745]: Source ID 2377 was not found when attempting to remove it
[badr@localhost ~]$ journalctl -u sshd.service -n 15
Nov 07 17:47:16 localhost.localdomain sshd[5609]: Server listening on 0.0.0.0 port 22.
Nov 07 17:47:16 localhost.localdomain sshd[5609]: Server listening on :: port 22.
Nov 07 17:47:16 localhost.localdomain systemd[1]: Started OpenSSH server daemon.
Nov 07 17:47:33 localhost.localdomain systemd[1]: Reloading OpenSSH server daemon...
Nov 07 17:47:33 localhost.localdomain systemd[1]: Reloaded OpenSSH server daemon.
Nov 07 17:47:33 localhost.localdomain sshd[5609]: Received SIGHUP; restarting.
Nov 07 17:47:33 localhost.localdomain sshd[5609]: Server listening on 0.0.0.0 port 22.
Nov 07 17:47:33 localhost.localdomain sshd[5609]: Server listening on :: port 22.
Nov 07 17:50:56 localhost.localdomain systemd[1]: sshd.service: Sent signal SIGTERM to main process 5609 (sshd) on client request.
Nov 07 17:50:56 localhost.localdomain sshd[5609]: Received signal 15; terminating.
Nov 07 17:50:56 localhost.localdomain systemd[1]: sshd.service: Deactivated successfully.
Nov 07 17:51:21 localhost.localdomain systemd[1]: Starting OpenSSH server daemon...
Nov 07 17:51:21 localhost.localdomain sshd[5890]: Server listening on 0.0.0.0 port 22.
Nov 07 17:51:21 localhost.localdomain sshd[5890]: Server listening on :: port 22.
Nov 07 17:51:21 localhost.localdomain systemd[1]: Started OpenSSH server daemon.
[badr@localhost ~]$
```

Figure 4.1 : Filtrage des logs du boot courant (-b) et ciblage exclusif de l'unité sshd.service (-u).

4.2 Surveillance en temps réel du flux d'événements (-f)

Commande exécutée :

```
journalctl -u sshd.service -f
```

Analyse et interprétation : Combinée avec le filtre d'unité, l'option -f (follow) maintient le curseur actif en fin de journal et affiche en direct toute nouvelle ligne de log produite par sshd.service, facilitant le débogage interactif lors des tests de connexion.

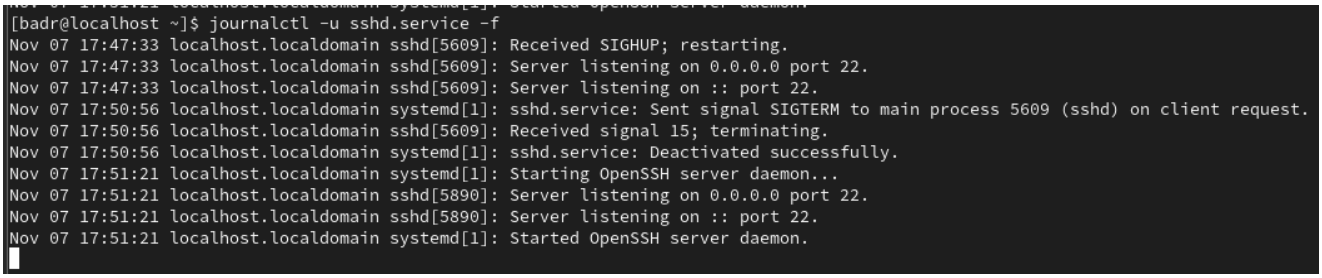


Figure 4.2 : Suivi continu en temps réel des journaux du service SSH avec journalctl -u sshd.service -f.

4.3 Filtrage par niveau de sévérité syslog (-p err / -p warning)

Commande exécutée :

```
journalctl -p err -n 10
journalctl -p warning -n 10
```

Analyse et interprétation : Le drapeau -p filtre les entrées selon les niveaux de priorité standards de syslog (de 0 emerg à 7 debug). Avec -p err (priorité 3 et plus graves), seules les erreurs critiques apparaissent en rouge (contrôleur SMBus piix4_smbus, initialisation audio alsactl, erreurs de session gdm-password). Avec -p warning (priorité 4 et plus graves), s'ajoutent en jaune les avertissements de synchronisation temporelle chronyd et du démon temps-réel rtkit-daemon.

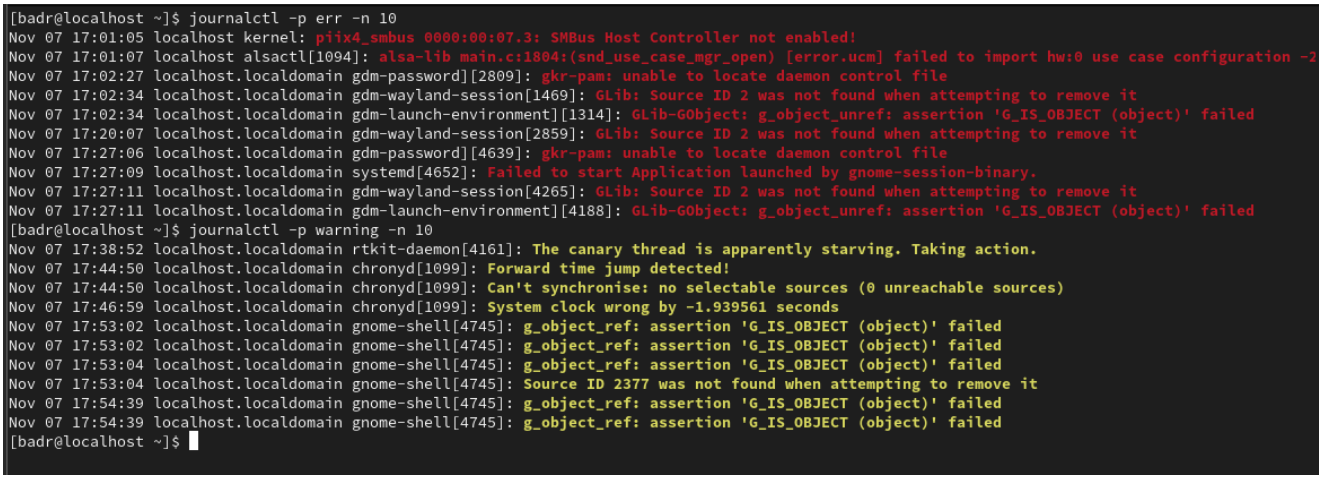


Figure 4.3 : Extraction ciblée des erreurs (-p err) et des avertissements (-p warning).

4.4 Filtrage temporel (--since) et limitation du nombre de lignes (-n)

Commande exécutée :

```
journalctl --since "today" -n 15
journalctl --since "17:00" -n 10
```

Analyse et interprétation : L'argument --since accepte des expressions relatives ("today") ou des horodatages précis ("17:00", "YYYY-MM-DD HH:MM:SS"). Associé à -n, il permet d'inspecter le début de la séquence d'initialisation du noyau Linux 5.14.0-687.61.1.el9_8.x86_64 et la cartographie mémoire physique BIOS-e820 enregistrée à 17:01:00.

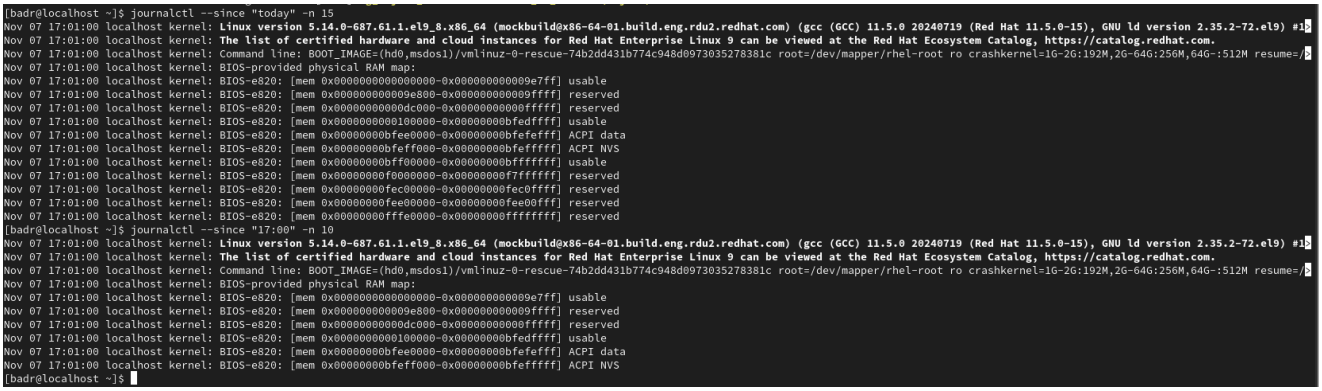


Figure 4.4 : Recherche temporelle dans les journaux avec --since "today" et --since "17:00".

4.5 Formats de sortie structurés (-o cat et -o json-pretty)

Commande exécutée :

```
journalctl -u sshd.service -o cat -n 5
journalctl -u sshd.service -o json-pretty -n 1
```

Analyse et interprétation : L'option -o modifie le formateur de sortie de journalctl. Le mode -o cat supprime toutes les colonnes d'en-tête (date, hôte, PID) pour ne conserver que la chaîne de texte brute, idéal pour les scripts shell. À l'inverse, le mode -o json-pretty restitue l'objet JSON complet stocké par journald avec l'ensemble de ses champs d'audit (_PID, _UID, _GID, _CMDLINE, _SYSTEMD_CGROUP, _SELINUX_CONTEXT, _BOOT_ID, PRIORITY, MESSAGE_ID), facilitant l'ingestion par des collecteurs SIEM ou Elasticsearch.

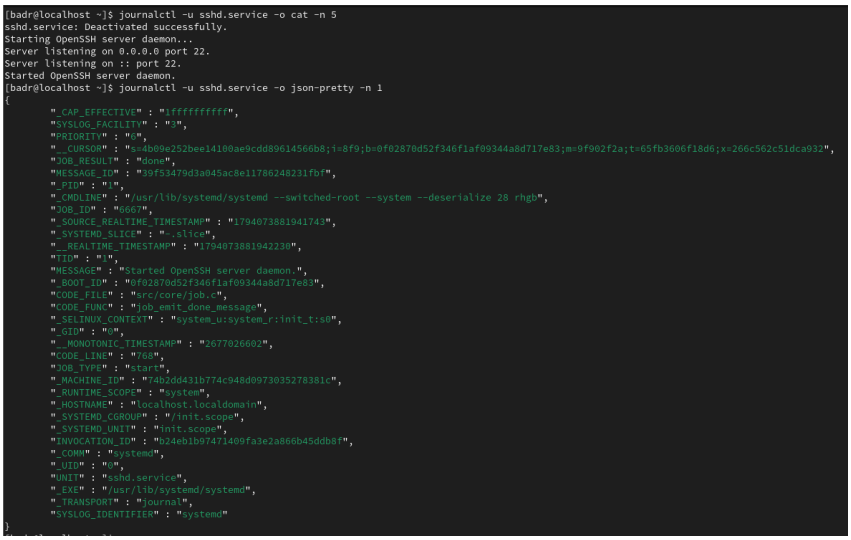


Figure 4.5 : Comparaison entre la sortie épurée (-o cat) et la structure complète (-o json-pretty).

Conclusion Générale

L'ensemble des manipulations réalisées au cours de ces quatre ateliers sur Red Hat Enterprise Linux 9.8 (Plow) a permis de maîtriser le fonctionnement interne de **systemd (v252)**. Nous avons vérifié son rôle central en tant que processus racine (PID 1), mesuré finement les performances d'amorçage avec **`systemd-analyze`** (19.215s au total), piloté le basculement à chaud entre les cibles **`graphical.target`** et **`multi-user.target`**, administré le cycle de vie complet des démons système (**`sshd`**, **`crond`**, **`httpd`**) et exploité les capacités de filtrage multicritères et d'export JSON de **`journalctl`** pour le diagnostic et la supervision.